Step 2 — The Three Extractor Agents — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — three chains, one input shape
- Hint 2 — which parser each one uses

Hint 1 — three chains, one input shape

Each agent is one small LCEL chain in its own file, and each takes the **same** input — `{"transcript": ...}` — so Step 3 can run them side by side:

- `src/agents/actions.py` → `build_actions_chain(model)`.
- `src/agents/decisions.py` → `build_decisions_chain(model)`.
- `src/agents/summary.py` → `build_summary_chain(model)`.

Each is `prompt | model(...)`. Keep each prompt short and single-purpose — the action prompt only mentions actions, the decision prompt only decisions. A focused prompt beats "extract everything at once."

Hint 2 — which parser each one uses

The output type decides the parser:

```
actions:   prompt | model.with_structured_output(ActionItems)   # typed list
decisions: prompt | model.with_structured_output(Decisions)     # typed list
summary:   prompt | model | StrOutputParser()                   # plain text
```

Decisions and actions need a *shape* your code loops over, so they use structured output. The summary is prose for a human, so `StrOutputParser` is right — don't force a schema onto free text. Each `build_*_chain(model)` returns a runnable you can `.invoke({"transcript": "..."})` on its own; the live run through all three is Step 3. The whole file set is in the Solution.